Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: ” Colecții: liste, tupluri, mulțimi, dicționare ”

A efectuat : st. gr.SI – 264 Șoimaru Gabriel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea A

A1. Scheletul și prima listă

Creează arsenal/privesc/enum.py. Întâi citești fișierele SUID într-o listă și o numeri. Un fișier de citire comun îți scoate liniile curate (fără goluri, fără comentarii).

A2. Nume unice: de la listă la mulțime

Numele fișierului e ce decide dacă e periculos (find, vim, nmap...). Scoate numele din fiecare cale și ține-le o singură dată.

A3. Constatările SUID: listă de dicționare

Fiecare SUID devine o fișă: calea, numele, riscul. Periculos = numele e într-o mulțime de binare cunoscute ca scăpări (stil GTFOBins). Întorci o listă de dicționare, sortată, fără dubluri.

A4. sudo și cron: tuplul și gruparea

O linie din sudo.txt are forma „user : drept". O despachetezi într-un tuplu. O linie din crontab.txt are șase câmpuri de program, apoi userul și comanda. Fiecare sursă dă o listă de dicționare.

A5. Raportul: JSON și rezumat pe categorii

main() cu argparse: primește folderul capturilor și un comutator --json. Cu --json scoate JSON curat; fără el, un rezumat pe categorii (cu .items()) și salvează raportul pe disc.

Lucrarea B

B1. Două mulțimi: curat și acum

Două mulțimi: mulțimea cailor din baza_suid.txt și mulțimea cailor din suid.txt

B2. Diferența: ce a apărut

Ce e în „acum" dar nu în „bază" sunt fișierele SUID noi. Un singur operator, nu o buclă dublă.

B3. Dicționar de risc și filtrul final

Faci o fișă nume -> risc pentru cele noi (dict comprehension), apoi filtrezi doar periculoasele (list comprehension cu if). Refolosești nume() și GTFO din enum printr-un import.

B4. Un test care prinde regresia

Scrie arsenal/privesc/test_diferenta.py cu un test mic, cu date scrise de mână, care verifică faptul cheie: ce e în acum și nu în bază iese corect. Rulează cu pytest și treacă fără rețea.

EFECTUAREA LUCRĂRII

Lucrarea A

A1

```python
#A1
import argparse, json, os
def citeste_linii(cale):
    linii = []
    with open(cale, encoding="utf-8") as f:
        for x in f:
            x = x.strip()
            if x and not x.startswith("#"):
                linii.append(x)
    return linii

cai = citeste_linii("arsenal/privesc/start/suid.txt")
print("linii SUID:", len(cai))
```

Figura 1.1.1 – Codul A1

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni> cd "Tehnici de programare\LabTP8"
>> python -m arsenal.privesc.enum
linii SUID: 14
```

Figura 1.1.2 – Rezultatul A1

A2

```python
#A2
def nume(cale):
    bucati = cale.rsplit("/", 1)
    rez = bucati[-1]
    return rez

nume_unice = {nume(c) for c in cai} # set comprehension
print("SUID distincte:", len(nume_unice))
```

Figura 1.2.1 – Codul A2

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.enum
SUID distincte: 13
```

Figura 1.2.2 – Rezultatul A2

A3

```python
GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
"less", "awk", "python3", "perl"}

def suid_constatari(cale):
    cai = citeste_linii(cale)
    lista = set(cai)
    rez = []
    for c in sorted(lista):
        x = {"cale": c, "nume": nume(c), "risc": "mare" if nume(c) in GTFO else "normal"}
        rez.append(x)
    return rez

suid = suid_constatari("arsenal/privesc/start/suid.txt")
mari = [c for c in suid if c["risc"] == "mare"]
print(len(suid), "SUID,", len(mari), "cu risc mare")
print(sorted(c["nume"] for c in mari))
```

Figura 1.3.1 – Codul A3

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.enum
 13 SUID, 4 cu risc mare
 ['cp', 'find', 'nmap', 'vim.basic']
```

Figura 1.3.2 – Rezultatul A3

A4

```python
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1) # despachetare in tuplu
        out.append({"user": user.strip(), "drept": drept.strip(), "risc": "mare" if "NOPASSWD" in drept else "normal"})
    return out

def cron_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split()
        user = campuri[5]
        comanda = " ".join(campuri[6:])
        rez = {"user": user, "comanda": comanda}
        out.append(rez)
    return out

def enumereaza(d):
    return {
    "suid": suid_constatari(os.path.join(d, "suid.txt")),
    "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
    "cron": cron_constatari(os.path.join(d, "crontab.txt")),
}

print(enumereaza("arsenal/privesc/start"))
```

Figura 1.4.1 – Codul A4

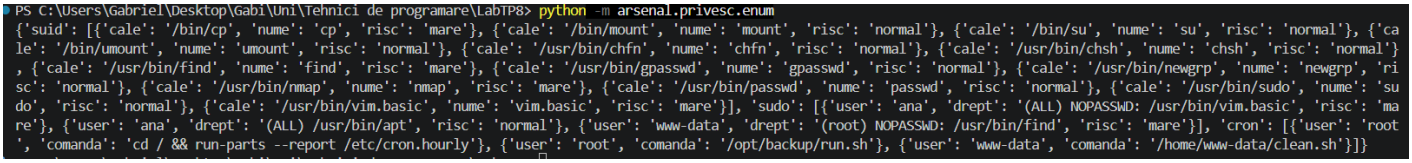


Figura 1.4.2 – Rezultatul A4

A5

```python
def main():
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    raport = enumereaza(a.capturi)
    if a.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False)); return
    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")
    users_sudo = {u["user"] for u in raport["sudo"]}
    users_cron = {u["user"] for u in raport["cron"]}
    users = users_sudo | users_cron
    print(f"utilizatorii implicati: {", ".join(sorted(users))}")
    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")

if __name__ == "__main__":
    main()
```

Figură 1.5.1 – Codul A5

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.enum arsenal/privesc/start
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatorii implicati: ana, root, www-data
Raport salvat in enum.json
```

Figura 1.5.2 – Rezultatul A5

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.enum arsenal/privesc/start --json
{
  "suid": [
    {
      "cale": "/bin/cp",
      "nume": "cp",
      "risc": "mare"
    },
    {
      "cale": "/bin/mount",
      "nume": "mount",
      "risc": "normal"
    },
    {
      "cale": "/bin/su",
      "nume": "su",
      "risc": "normal"
```

Figura 1.5.3 – Fragment din conținutul fișierului enum.json

Lucrarea B

B1

```python
from arsenal.privesc.enum import citeste_linii, nume, GTFO

#B1
def cai(fisier):
    out = []
    for x in citeste_linii(fisier):
        out.append(x)
    return set(out)
```

Figura 2.1.1 – Codul B1

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.diferenta
baza: 9  acum: 13
```

Figura 2.1.2 – Rezultatul B1

B2

```python
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for c in sorted(noi):
    print(" ", c)
```

Figura 2.2.1 – Codul B2

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.diferenta
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

Figura 2.2.2 – Rezultatul B2

B3

```
risc = {nume(c): ("mare" if nume(c) in GTFO else "normal") for c in noi}
periculoase = [c for c in sorted(noi) if nume(c) in GTFO]
print("periculoase NOI:", periculoase)
```

Figura 2.3.1 – Codul B3

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> python -m arsenal.privesc.diferenta
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic']
```

Figură 2.3.2 – Rezultatul B3

B4

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}
```

Figura 2.4.1 – Codul B4

```
PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP8> pytest -q arsenal/privesc/test_diferenta.py
.
1 passed in 0.01s
```

Figura 2.4.2 – Rezultatul B4

CONCLUZIE

În această lucrare am folosit metode de lucru cu colecțiile: liste, dicționare, mulțimi (seturi) și tupluri. Am folosit split() și rsplit() pentru a transforma un string într-o listă, append() pentru a adăuga un element în listă, sorted() pentru a crea o listă nouă din colecția oferită, reuniunea și diferența mulțimilor.